

YOLO 기반 UI 요소 탐지 기능 구현 보고서

문서 개요

본 보고서는 D:\WappWyolo 폴더에 추가한 YOLO 기반 UI 요소 탐지 스크립트에 대한 구현 내용을 정리한 문서이다.

이번 작업의 목표는 스크린샷 이미지를 입력으로 받아, 로컬에 저장된 Ultralytics YOLO .pt 모델을 사용해 UI 요소 후보를 탐지하고, 탐지 결과를 사람이 확인할 수 있는 형태로 출력하는 것이다.

구현 결과물은 다음 요구사항을 만족한다.

- Python 사용
- Ultralytics YOLO 사용
- 로컬 .pt 모델 로드
- 스크린샷 이미지 입력
- bounding box 좌표 출력
- class name 출력
- confidence score 출력
- bounding box가 그려진 이미지 저장
- 선택적으로 JSON 결과 파일 저장

1. 작업 목적

1.1 배경

GUI 자동화나 화면 분석 작업에서는 스크린샷 안에 어떤 UI 요소가 존재하는지 파악하는 단계가 중요하다. 기존의 UIA, OCR, 좌표 기반 방식은 각각 장점이 있지만, 다음과 같은 상황에서는 이미지 기반 탐지가 도움이 된다.

- UIA 접근성 정보가 부족한 프로그램
- 커스텀 렌더링으로 인해 버튼이나 입력 필드가 표준 컨트롤로 잡히지 않는 경우
- 화면 이미지 기준으로 영역을 빠르게 표시하고 싶은 경우
- OCR 이전에 텍스트 영역 후보를 먼저 찾고 싶은 경우

이번 YOLO 스크립트는 이러한 이미지 기반 UI 탐지 흐름을 검증하기 위한 최소 실행 단위로 작성되었다.

1.2 목표

이번 구현의 직접적인 목표는 다음과 같다.

- 스크린샷 이미지
 - > 로컬 YOLO 모델 추론
 - > 탐지 결과 좌표/클래스/신뢰도 출력
 - > bounding box 시각화 이미지 저장

이 기능은 이후 GUI 자동화 프레임워크에서 화면 기반 인식 보조 모듈로 확장할 수 있다.

2. 구현 위치

이번 작업은 기존 파일을 수정하지 않고 yolo 폴더를 새로 만들어 독립적으로 구성했다.

```
D:\app\yolo
detect_ui_elements.py
requirements.txt
README.md
report\
  yolo_ui_detection_report.md
```

각 파일의 역할은 다음과 같다.

파일	역할
detect_ui_elements.py	YOLO 모델 로드, 이미지 추론, 결과 출력, 시각화 이미지 저장
requirements.txt	실행에 필요한 Python 패키지 목록
README.md	설치 및 실행 방법 안내
report/yolo_ui_detection_report.md	구현 내용과 활용 방안 정리 보고서

3. 실행 환경

3.1 기본 환경

운영체제: Windows
작업 경로: D:\Wapp
언어: Python
주요 라이브러리: ultralytics, opencv-python
입력 모델: 로컬 .pt 파일
입력 이미지: 스크린샷 이미지

3.2 의존성

requirements.txt에는 다음 패키지를 정의했다.

```
ultralytics  
opencv-python
```

설치 명령은 다음과 같다.

```
pip install -r yolo\requirements.txt
```

4. 스크립트 동작 방식

4.1 전체 흐름

detect_ui_elements.py의 전체 실행 흐름은 다음과 같다.

1. 명령행 인자 파싱
2. 모델 파일 경로 검증
3. 입력 이미지 파일 경로 검증
4. Ultralytics YOLO 모델 로드
5. 스크린샷 이미지 추론 실행
6. 각 detection의 좌표, 클래스, confidence 추출
7. bounding box가 그려진 이미지 생성
8. 결과 이미지 저장
9. detection 결과를 JSON 형태로 콘솔 출력
10. --json-output 옵션이 있으면 JSON 파일 저장

4.2 입력 인자

옵션	필수 여부	설명
--model	필수	사용할 로컬 YOLO .pt 모델 경로
--image	필수	탐지할 스크린샷 이미지 경로
--output	선택	bounding box 결과 이미지 저장 경로
--json-output	선택	탐지 결과 JSON 저장 경로
--conf	선택	confidence threshold
--imgsz	선택	추론 이미지 크기

4.3 실행 예시

```
python yolo\detect_ui_elements.py --model path\to\ui-model.pt --image path\to\screenshot.png  
--output yolo\result.jpg --json-output yolo\detections.json
```

5. 출력 데이터 구조

5.1 콘솔 출력

스크립트는 탐지 결과를 JSON 형태로 출력한다.

```
{
  "image": "D:\\Wapp\\sample\\screenshot.png",
  "model": "D:\\Wapp\\models\\ui-model.pt",
  "output_image": "D:\\Wapp\\yolo\\result.jpg",
  "detections": [
    {
      "bbox": {
        "x1": 120.5,
        "y1": 82.0,
        "x2": 210.2,
        "y2": 118.7
      },
      "class_id": 0,
      "class_name": "button",
      "confidence": 0.9321
    }
  ]
}
```

5.2 detection 필드 설명

필드	설명
bbox.x1	bounding box 좌측 상단 X 좌표
bbox.y1	bounding box 좌측 상단 Y 좌표
bbox.x2	bounding box 우측 하단 X 좌표
bbox.y2	bounding box 우측 하단 Y 좌표
class_id	모델 내부 클래스 ID
class_name	모델이 제공하는 클래스 이름
confidence	탐지 신뢰도

5.3 결과 이미지

result.plot()을 사용해 Ultralytics가 제공하는 기본 시각화 이미지를 생성한다. 이후 OpenCV의 cv2.imwrite()로 지정한 경로에 저장한다.

저장 이미지는 다음 정보가 포함된다.

- bounding box
- class label

- confidence

6. 구현 상세

6.1 모델 로드

YOLO 모델은 다음 방식으로 로드한다.

```
model = YOLO(str(model_path))
```

model_path는 반드시 로컬 .pt 파일이어야 한다. 이 방식은 네트워크 다운로드에 의존하지 않고, 사용자가 제공한 모델만 사용한다.

6.2 추론 실행

추론은 다음 방식으로 실행한다.

```
results = model.predict(
    source=str(image_path),
    conf=args.conf,
    imgsz=args.imgsz,
    verbose=False,
)
```

conf와 imgsz는 명령행 옵션으로 조정할 수 있도록 했다. 따라서 모델이나 스크린샷 특성에 따라 민감도를 조정할 수 있다.

6.3 결과 변환

Ultralytics 결과 객체의 result.bboxes를 순회하면서 다음 정보를 추출한다.

```
class_id = int(box.cls[0].item())
x1, y1, x2, y2 = [float(value) for value in box.xyxy[0].tolist()]
confidence = float(box.conf[0].item())
```

좌표는 xyxy 형식을 사용한다.

```
x1, y1: 좌측 상단 좌표
x2, y2: 우측 하단 좌표
```

7. 주의사항과 한계

7.1 모델 클래스 의존성

YOLO의 class name은 스크립트가 임의로 만드는 값이 아니라 .pt 모델 안에 저장된 클래스 메타데이터를 따른다.

따라서 button, text, input_field 같은 결과를 얻으려면 해당 UI 클래스로 학습된 모델이 필요하다.

일반적인 COCO 사전 학습 모델은 다음과 같은 일반 객체 클래스 중심이다.

person, car, chair, bottle, keyboard ...

따라서 일반 COCO 모델을 그대로 사용하면 UI 버튼이나 입력 필드를 원하는 클래스명으로 탐지하지 못할 수 있다.

7.2 OCR과의 관계

YOLO는 영역 탐지 모델이므로 텍스트 내용을 읽는 기능은 제공하지 않는다.

예를 들어 text 클래스를 탐지할 수는 있지만, 그 안에 적힌 실제 문자열을 읽으려면 OCR 모듈과 함께 사용해야 한다.

향후 구조는 다음처럼 확장할 수 있다.

```
YOLO로 텍스트 영역 탐지
-> 탐지된 영역 crop
-> OCR 실행
-> 텍스트 내용 추출
```

7.3 UIA와의 관계

YOLO 기반 탐지는 화면 이미지 기준 방식이다. 따라서 UIA처럼 control tree, automation id, name 같은 구조적 정보를 제공하지 않는다.

대신 UIA가 약한 화면에서도 시각적 후보 영역을 찾을 수 있다는 장점이 있다.

8. 검증 내용

현재 작업에서는 스크립트 문법 검증을 수행했다.

```
python -m py_compile yolo\detect_ui_elements.py
```

검증 결과:

성공

다만 실제 YOLO 추론은 로컬 .pt 모델과 테스트용 스크린샷 이미지가 필요하므로, 현재 단계에서는 실행하지 않았다.

실제 추론 검증을 위해서는 다음 파일이 필요하다.

- UI 요소 탐지용 .pt 모델
- 테스트 스크린샷 이미지

추가로 현재 단계에서는 실제 실행용 스크립트를 다음 파일로 정리했다.

```
D:\app\yolo\detect_ui.py  
D:\app\yolo\train_ui_yolo.py  
D:\app\yolo\dataset\data.yaml
```

따라서 모델과 테스트 이미지가 준비되면 다음 명령으로 실제 추론을 수행할 수 있다.

```
python yolo\detect_ui.py --model yolo\models\ui_model.pt --image yolo\screenshots\test.png  
--conf 0.25 --imgsz 1280 --out yolo\outputs
```

Salesforce GPA-GUI-Detector를 사용할 경우의 예시는 다음과 같다.

```
python yolo\detect_ui.py --model yolo\models\gpa-gui-detector\model.pt --image  
yolo\screenshots\test.png --conf 0.05 --iou 0.1 --imgsz 1280 --out yolo\outputs
```

MacPaw Screen2AX UI Elements 모델을 사용할 경우의 예시는 다음과 같다.

```
python yolo\detect_ui.py --model yolo\models\ui-elements-detection.pt --image  
yolo\screenshots\test.png --conf 0.25 --iou 0.7 --imgsz 1280 --out yolo\outputs
```

9. 공개 모델 후보

9.1 MacPaw YOLO UI Elements Detection

macpaw-research/yolov11l-ui-elements-detection는 Hugging Face에 공개된 UI element detection 모델이다.

주요 특징은 다음과 같다.

- 파일명: ui-elements-detection.pt
- 기반: Ultralytics YOLO11 계열
- 대상: macOS 애플리케이션 스크린샷
- 대표 클래스: AXButton, AXDisclosureTriangle, AXImage, AXLink, AXTextArea
- 라이선스: AGPL-3.0

이 모델은 macOS accessibility element 관점의 클래스명을 사용하므로, Windows GUI 자동화에서 원하는 button, text, input_field와 이름이 완전히 일치하지 않을 수 있다. 다만 UI 요소 탐지 baseline으로는 유용하다.

9.2 Salesforce GPA-GUI-Detector

Salesforce/GPA-GUI-Detector는 GUI Process Automation을 위한 YOLO 기반 detector이다.

주요 특징은 다음과 같다.

- 파일명: model.pt
- 기반: Ultralytics YOLO
- 대상: GUI element detection
- 활용 목적: 화면 기반 GUI 자동화
- 라이선스: MIT

이 모델은 GUI 자동화 목적과 직접적으로 맞닿아 있으므로 현재 실험의 우선 후보로 볼 수 있다.

9.3 Roboflow 데이터셋 또는 모델

Roboflow의 공개 UI element detection 프로젝트는 클래스 정의와 데이터 분포가 프로젝트마다 다르다.

따라서 사용 전 다음 항목을 확인해야 한다.

- 목표 클래스 포함 여부
- YOLO export 지원 여부
- 라이선스
- 데스크톱 GUI, 웹 UI, 모바일 UI 중 어떤 화면에 가까운지

10. Fine-tuning 설계

10.1 데이터 수집

소콘 프로그램 또는 자동화 대상 프로그램을 다양한 상태로 실행하고 스크린샷을 수집한다.

수집해야 할 화면 예시는 다음과 같다.

- 기본 화면
- 버튼 hover 또는 focus 상태
- 입력 필드 focus 상태
- 체크박스 on/off 상태
- 드롭다운 열린 상태
- 팝업 또는 알림 표시 상태
- 에러 메시지 표시 상태
- 서로 다른 해상도와 테마

10.2 라벨링

라벨링 도구는 다음 후보를 사용할 수 있다.

- LabelImg
- CVAT
- Roboflow
- Label Studio

출력 형식은 YOLO object detection format으로 맞춘다.

10.3 클래스 정의

초기 클래스는 다음처럼 정의한다.

```
0 button
1 text
2 input_field
3 checkbox
4 dropdown
5 icon
6 popup
```

10.4 Dataset 구조

```
yolo/dataset/
```

```
images/  
  train/  
  val/  
  test/  
labels/  
  train/  
  val/  
  test/  
data.yaml
```

data.yaml 예시는 D:\app\yolo\dataset\data.yaml에 추가했다.

10.5 학습 명령

```
python yolo\train_ui_yolo.py --data yolo\dataset\data.yaml --model yolo\models\yolo11n.pt  
--epochs 100 --imgsz 1280 --batch 8 --device 0 --project yolo\runs --name ui_yolo
```

CPU 환경에서는 다음처럼 실행할 수 있다.

```
python yolo\train_ui_yolo.py --data yolo\dataset\data.yaml --model yolo\models\yolo11n.pt  
--epochs 50 --imgsz 960 --batch 2 --device cpu --project yolo\runs --name ui_yolo_cpu
```

10.6 검증 명령

```
python yolo\train_ui_yolo.py --val-only --data yolo\dataset\data.yaml --model  
yolo\runs\ui_yolo\weights\best.pt --imgsz 1280 --batch 8 --device 0
```

11. 폐쇄망 환경 고려

폐쇄망에서는 코드가 인터넷에 접근하지 않도록 로컬 .pt 파일만 사용해야 한다.

외부망 PC에서 준비할 항목은 다음과 같다.

```
offline_packages/  
yolo/models/*.pt  
테스트용 screenshot.png
```

외부망 PC에서 wheel 파일을 다운로드한다.

```
pip download -r yolo\requirements.txt -d offline_packages
```

내부망 PC에서는 다음 방식으로 설치한다.

```
pip install --no-index --find-links offline_packages -r yolo\requirements.txt
```

이후 내부망에서는 다음처럼 로컬 모델만 지정해서 실행한다.

```
python yolo\detect_ui.py --model yolo\models\gpa-gui-detector\model.pt --image  
yolo\screenshots\test.png --conf 0.05 --iou 0.1 --imgsz 1280 --out yolo\outputs
```


12. 향후 확장 방향

12.1 GUI 자동화 시스템과 연동

탐지 결과의 bounding box 중심 좌표를 계산하면 자동 클릭 후보로 사용할 수 있다.

예시:

```
center_x = (x1 + x2) / 2  
center_y = (y1 + y2) / 2
```

이를 기존 자동화 runner의 click action과 연결하면 화면 기반 클릭 기능으로 확장할 수 있다.

12.2 OCR 연동

text 클래스 탐지 결과를 crop한 뒤 OCR에 전달하면 화면의 텍스트 영역을 더 정밀하게 읽을 수 있다.

이 방식은 전체 화면 OCR보다 탐색 범위를 줄일 수 있어 속도와 정확도 측면에서 유리할 수 있다.

12.3 결과 포맷 표준화

현재 JSON 구조는 간단한 형태로 구성되어 있다. 이후 자동화 시스템과 직접 연결하려면 다음 필드를 추가할 수 있다.

- detection id
- center point
- width, height
- normalized coordinate
- source image size
- timestamp
- model metadata

13. 결론

이번 작업으로 D:WappWyolo 폴더 안에 YOLO 기반 UI 요소 탐지 기능의 최소 실행 구조를 마련했다.

스크립트는 로컬 .pt 모델과 스크린샷 이미지를 입력받아 탐지 결과를 JSON 형태로 출력하고, bounding box가 그려진 결과 이미지를 저장한다.

현재 구현은 독립 실행 가능한 POC 형태이며, 향후 UIA 기반 자동화, OCR, 클릭 좌표 생성 로직과 연결하면 이미지 기반 GUI 자동화 보조 모듈로 확장할 수 있다.

실제 추론을 위해 남은 조건은 다음 두 가지이다.

- UI 요소 탐지용 로컬 .pt 모델 준비
- 테스트용 스크린샷 이미지 준비

14. 실제 추론 실행 결과

14.1 실행 일시와 목적

2026-05-05에 실제 공개 YOLO UI detection 모델을 다운로드하여 로컬 추론을 수행했다.

이번 실행의 목적은 다음과 같다.

- 공개 UI element detection 모델을 로컬 .pt 파일로 준비
- 기존 GUI 자동화 스크린샷을 입력 이미지로 사용
- detect_ui.py가 실제로 JSON과 bounding box 이미지를 생성하는지 확인
- 모델별 클래스 출력 차이를 비교

14.2 테스트 입력 이미지

테스트 이미지는 기존 profile generator 산출물 중 하나를 복사해 사용했다.

원본: D:\Wapp\profiles\generated\wfc_desktop_2026-05-03_151014\sscreenshot.png
복사본: D:\Wapp\yolo\sscreenshots\wtest.png

이미지 크기는 다음과 같다.

width: 1440
height: 1353

14.3 설치 중 발생한 문제와 해결

처음 ultralytics와 huggingface_hub를 설치할 때 sandbox 내부 네트워크 제한으로 PyPI 접근이 실패했다.

이후 권한 승인을 받아 외부 네트워크로 설치를 다시 수행했다.

```
python -m pip install --user ultralytics huggingface_hub
```

설치 후 최초 PyTorch 조합에서 다음 문제가 발생했다.

```
OSError: [WinError 1114] DLL 초기화 루틴을 실행할 수 없습니다.  
Error loading ... torch\lib\torch10.dll
```

해결을 위해 Windows CPU 환경에서 안정적으로 동작한 조합으로 재설치했다.

```
python -m pip install --user --force-reinstall torch==2.5.1 torchvision==0.20.1
```

이 과정에서 numpy 2.x가 설치되며 기존 OpenCV와 충돌했다.

```
ImportError: numpy.core.multiarray failed to import
```

OpenCV가 정상 import 되도록 numpy 1.26.4로 되돌렸다.

```
python -m pip install --user --force-reinstall numpy==1.26.4
```

최종적으로 동작한 주요 패키지 조합은 다음과 같다.

```
torch==2.5.1
torchvision==0.20.1
numpy==1.26.4
opencv-python>=4.9,<5
ultralytics
huggingface_hub
```

이 조합은 D:\Wapp\yolo\requirements.txt에 반영했다.

14.4 Salesforce GPA-GUI-Detector 실행 결과

먼저 Salesforce의 GPA-GUI-Detector 모델을 다운로드했다.

```
python -c "from huggingface_hub import hf_hub_download;
p=hf_hub_download(repo_id='Salesforce/GPA-GUI-Detector', filename='model.pt',
local_dir='yolo/models/gpa-gui-detector'); print(p)"
```

모델 경로:

```
D:\Wapp\yolo\models\gpa-gui-detector\model.pt
```

확인된 클래스는 다음과 같다.

```
{0: "icon"}
```

실행 명령:

```
python yolo\detect_ui.py --model yolo\models\gpa-gui-detector\model.pt --image
yolo\screenshots\test.png --conf 0.05 --iou 0.1 --imgsz 1280 --out yolo\outputs --device cpu
```

출력 파일:

```
D:\Wapp\yolo\outputs\test_20260505_181741_annotated.png
D:\Wapp\yolo\outputs\test_20260505_181741_detections.json
```

결과 요약:

```
detections_count: 65
class distribution:
  icon: 65
```

해석:

Salesforce 모델은 GUI 자동화 목적의 후보 영역 탐지에는 사용할 수 있으나, 클래스가 icon 하나뿐이므로 button, text, input_field를 구분하는 목적에는 맞지 않는다.

14.5 MacPaw YOLO UI Elements Detection 실행 결과

다음으로 MacPaw의 UI elements detection 모델을 다운로드했다.

```
python -c "from huggingface_hub import hf_hub_download; p=hf_hub_download(repo_id='macpaw-research/yolov11l-ui-elements-detection', filename='ui-elements-detection.pt', local_dir='yolo/models/macpaw-ui-elements'); print(p)"
```

모델 경로:

D:\Wapp\yolo\models\macpaw-ui-elements\ui-elements-detection.pt

확인된 클래스는 다음과 같다.

```
{
  0: "AXButton",
  1: "AXDisclosureTriangle",
  2: "AXImage",
  3: "AXLink",
  4: "AXTextArea"
}
```

실행 명령:

```
python yolo\detect_ui.py --model yolo\models\macpaw-ui-elements\ui-elements-detection.pt
--image yolo\screenshots\test.png --conf 0.25 --iou 0.7 --imgsz 1280 --out yolo\outputs
--device cpu
```

출력 파일:

D:\Wapp\yolo\outputs\test_20260505_182201_annotated.png
D:\Wapp\yolo\outputs\test_20260505_182201_detections.json

결과 요약:

```
detections_count: 81
class distribution:
  AXButton: 46
  AXTextArea: 34
  AXLink: 1
```

해석:

MacPaw 모델은 Salesforce 모델보다 이번 목적에 더 적합했다. 버튼 후보는 AXButton, 텍스트 또는 입력 영역 후보는 AXTextArea로 분리되어 출력되었다.

다만 이 모델은 macOS accessibility naming을 사용하므로 Windows GUI 자동화에서 원하는 최종 클래스명과는 차이가 있다.

권장 후처리 매핑은 다음과 같다.

```
AXButton -> button
AXTextArea -> text 또는 input_field 후보
AXLink -> link
AXImage -> image 또는 icon
AXDisclosureTriangle -> dropdown 또는 disclosure
```

AXTextArea를 text와 input_field로 더 나누려면 OCR, UIA 정보, 박스 크기, 주변 label 위치, focus 가능 여부 같은 추가 신호가 필요하다.

14.6 최종 판단

두 공개 모델 비교 결과는 다음과 같다.

모델	클래스 구조	결과	판단
Salesforce GPA-GUI-Detector	icon 단일 클래스	65개 탐지, 전부 icon	클릭 후보 탐지용 baseline
MacPaw UI Elements Detection	AXButton, AXTextArea 등 5개 클래스	81개 탐지, 버튼/텍스트 후보 분리	현재 목적에 더 적합

따라서 현 단계에서는 MacPaw 모델을 기본 실험 모델로 사용하는 것이 좋다.

단, 최종 목표가 button, text, input_field, checkbox, dropdown, icon, popup처럼 Windows GUI 자동화에 맞는 클래스라면 직접 수집한 소콘 프로그램 스크린샷으로 fine-tuning 하는 절차가 필요하다.

15. 버전 업데이트

이번 YOLO 실험 추가는 다음 내용을 포함한다.

- detect_ui.py 실제 추론 스크립트 추가
- train_ui_yolo.py fine-tuning 보조 스크립트 추가
- dataset/data.yaml 예시 추가
- Salesforce GPA-GUI-Detector 모델 실행 검증
- MacPaw UI Elements Detection 모델 실행 검증
- 실제 추론 결과 JSON과 annotated image 생성 확인
- 한국어 README와 보고서 정리
- PDF 보고서 생성

이에 따라 프로젝트 버전은 다음과 같이 갱신한다.

이전 버전: v1.2.0
신규 버전: v1.3.0